

# ROS 2 Core Overview

This is meant to explain ROS 2 core basics, design, and features.

Our centralized mainline documentation site is [ROS 2 documentation](#) if we need to know more details.

# Agenda

- Governance
- Distributions
- Architecture / Design
- Basic Concepts
- Features



# Governance

## Open Source Robotics Alliance

is a new initiative of the OSRF to organize and strengthen project governance and community involvement. The OSRA is a mixed membership and meritocratic model, following other successful foundations for open-source projects.



## ROS Project Management Committee

The ROS Project Management Committee is responsible for the day-to-day operations of the ROS 2 project. The ROS PMC consists of the Project Leader, the ROS PMC Members (who have full voting rights)

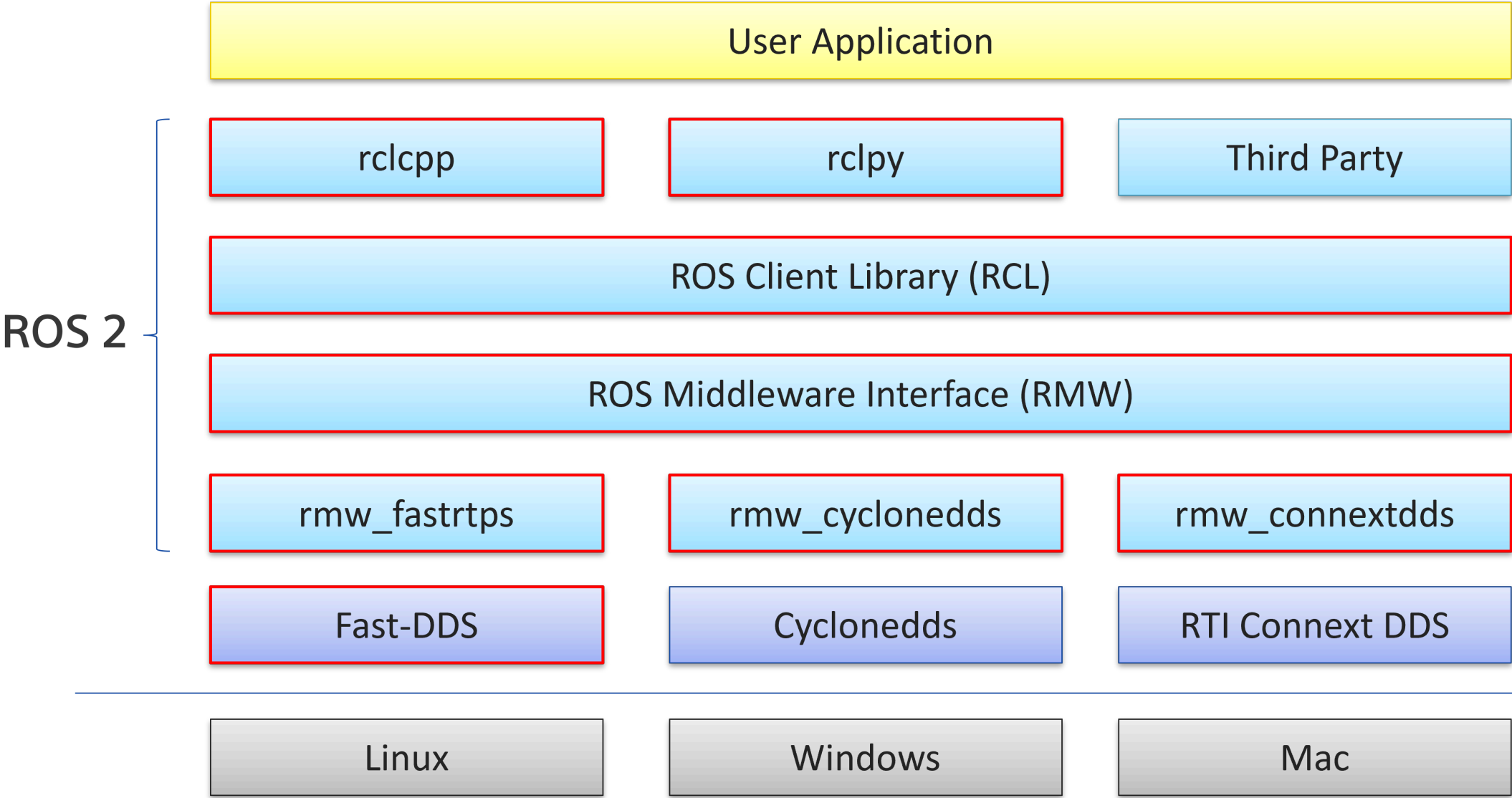
See [Current ROS PMC Constituents](#).

## Community Sites

- [ROS 2 Github](#)
  - Development, Design, Source code
- [ROS Discourse](#)
  - Discussion / Announcement / Release
- [Robotics Stack Exchange](#)
  - Centralized QA platform
- Working Group / Specific Interest Group
  - Under construction since OSRA structure transition.

# Distributions

| Distribution Name | Release Date   | End Of Life Date |
|-------------------|----------------|------------------|
| Lyrical Luth      | May 2026       | May 2031         |
| Kilted Kaiju      | May 2025       | November 2026    |
| Jazzy Jalisco     | May 23rd, 2024 | May 2029         |
| Iron Irwini       | May 23rd, 2023 | November 2024    |
| Humble Hawksbill  | May 23rd, 2022 | May 2027         |



# Data Distribution Service

The **OMG Data-Distribution Service for Real-Time Systems® (DDS®)** is the first open international middleware standard directly addressing publish-subscribe communications for real-time and embedded systems.

DDS introduces a virtual **Global Data Space** where applications can share information by simply reading and writing **data-objects** addressed by means of an application-defined name (Topic) and a key. DDS features fine and extensive control of **QoS** parameters, including reliability, bandwidth, delivery deadlines, and resource limits. DDS also supports the construction of **local object models** on top of the Global Data Space.



# Zenoh

New Tier 1 RMW implementation.

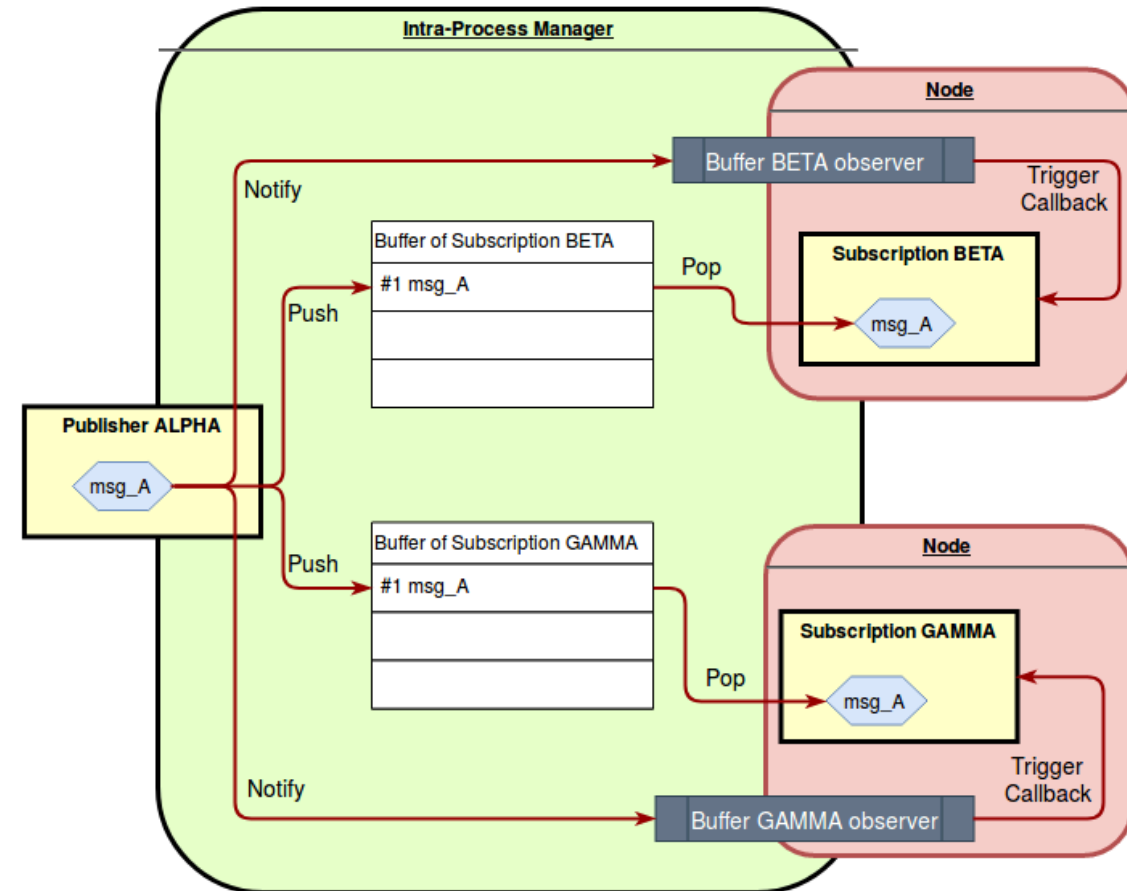
Zenoh (pronounced /zeno/) unifies data in motion, data at rest, and computations. It carefully blends traditional pub/sub with geo-distributed storage, queries, and computations, while retaining a level of time and space efficiency that is well beyond any of the mainstream stacks.

# Basic Concepts

- [Node](#) / [LifecycleNode](#) / [Components](#)
- [Executor](#)
- [Topics](#), [Services](#), [Parameters](#) and [Actions](#)
- [Logging](#)

# Node

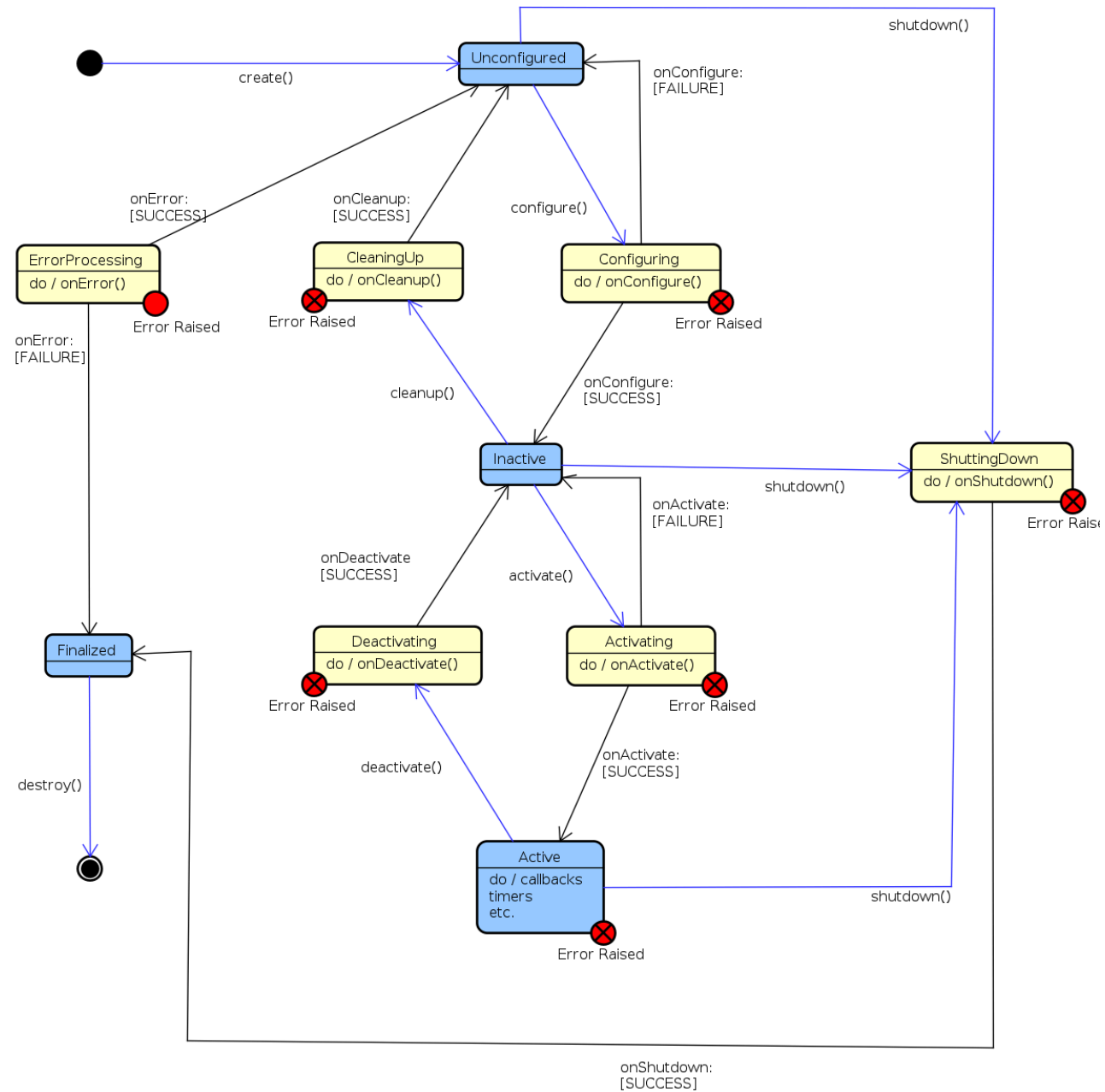
- The most basic instance of ROS 2
- Single process can be constructed with multiple Nodes
- Intra-Process Manager for most efficient data transmission



# LifeCycleNode

A managed life cycle for nodes allows greater control over the state of ROS system. It allows `roslaunch` to ensure that all components have been instantiated correctly before it allows any component to begin executing its behavior. It will also allow nodes to be restarted or replaced on-line.

- Primary State
  - `Unconfigure`, `Inactive`, `Active` and `Finalized`
- Transition State
  - `Configuring`, `CleanningUp`, `ShuttingDown`, `Activating`, `Deactivating` and `ErrorProcessing`



# Components

- inherited/inspired from ROS 1 Nodelet
- Multiple components in a single process
- Can load and unload the composable node
- Composable nodes as shared libraries

# Executor

- SingleThreadedExecutor: single thread to dispatch task and execute.
- MultiThreadedExecutor: mulith-thread to dispatch task and execute. (via `std::thread::hardware_concurrency` )
- EventExecutor (rclcpp): Uses events queue in the Executor to execute the entities from associated nodes.
- EventsCBGExecutor (rclcpp): [Callback Group Events Executor](#), like the `EventsExecutor` it uses an events queue, but adds support for multiple sources of ROS time and multiple threads. Uses 10% to 15% less CPU than the Single/Multi-Threaded executors. (available on Lyrical Luth)

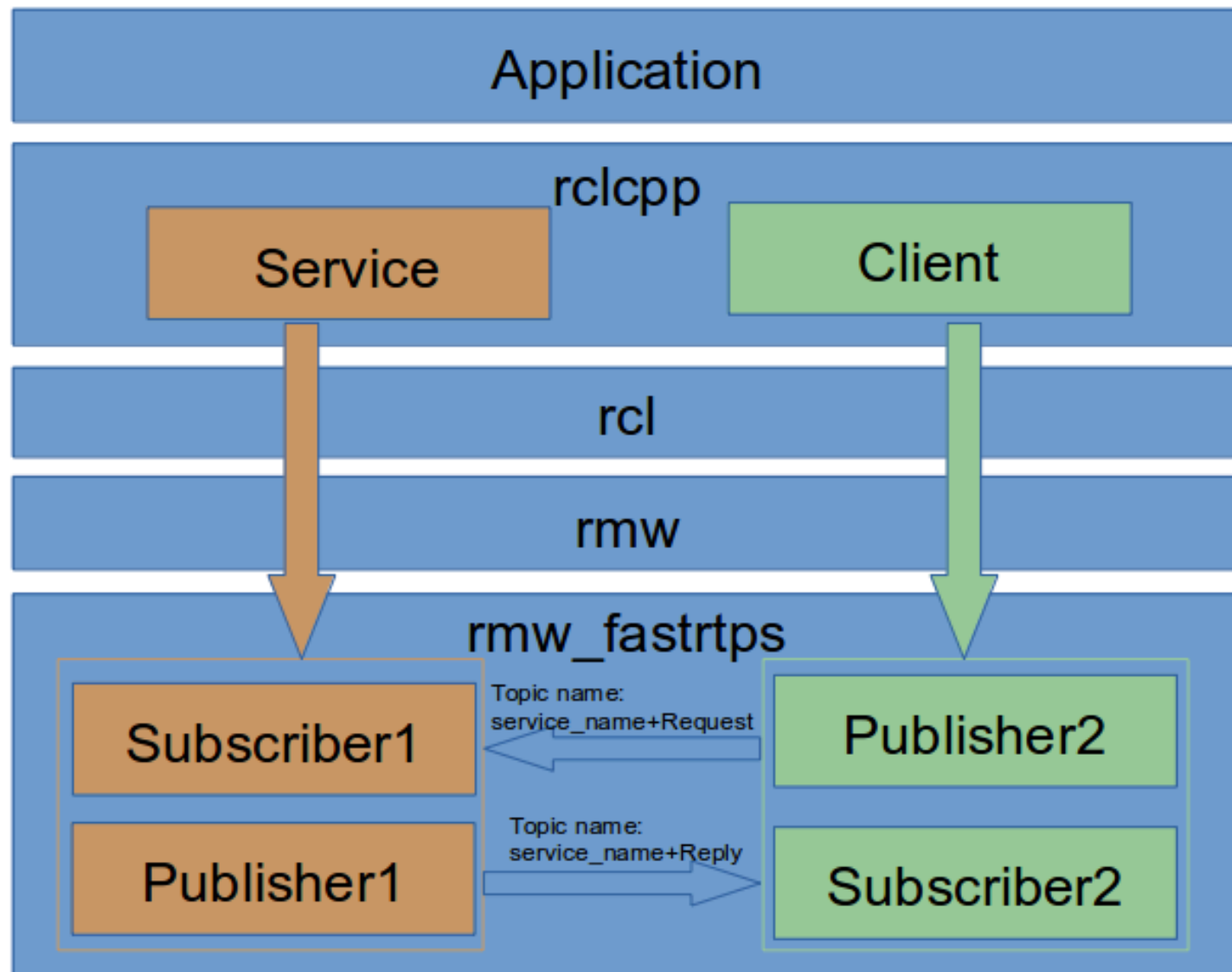
# Topics

 ros2 topic



# Service

 ros2 service

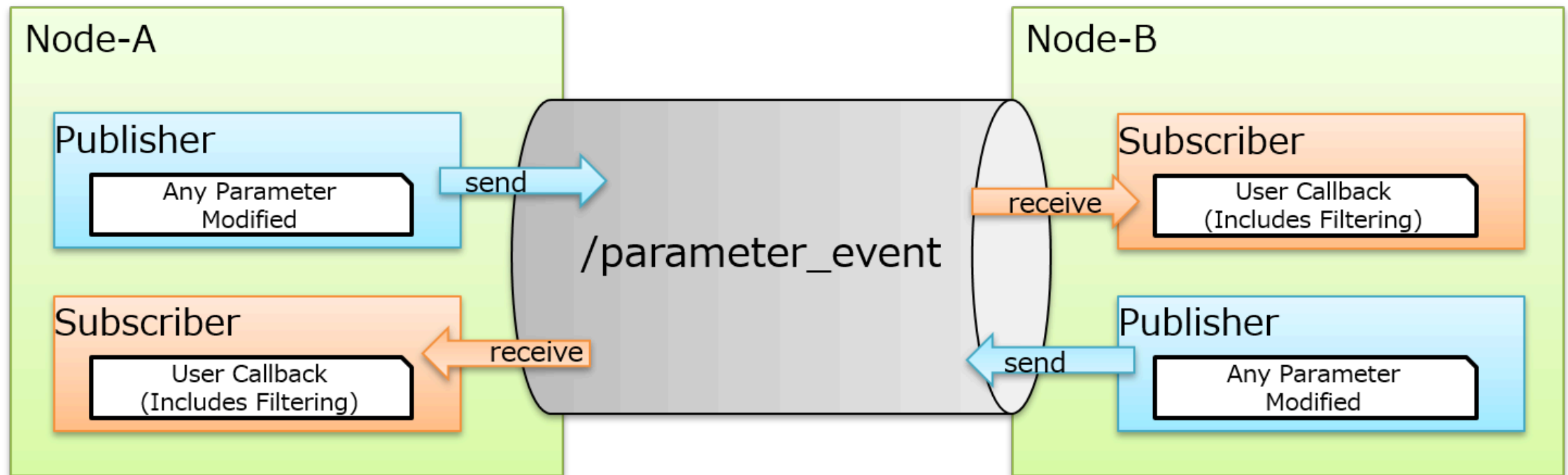


Service architecture

# Parameter

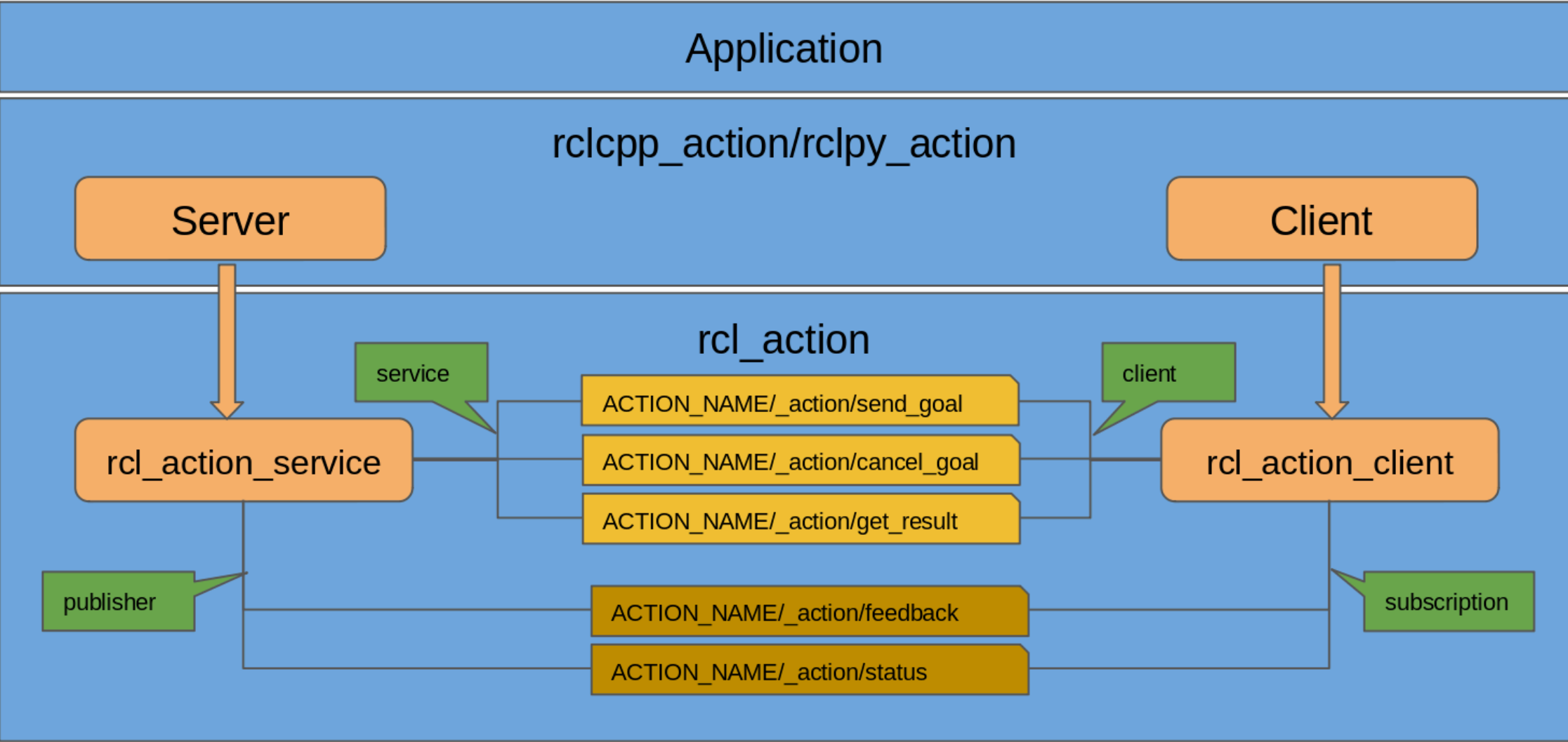
- No parameter server
- Parameters are hosted in Node
- Async / Sync Parameter Client
- `get` , `set` , `list` , `delete` and `describe` methods
- user callback to validate parameter change

# Parameter Events

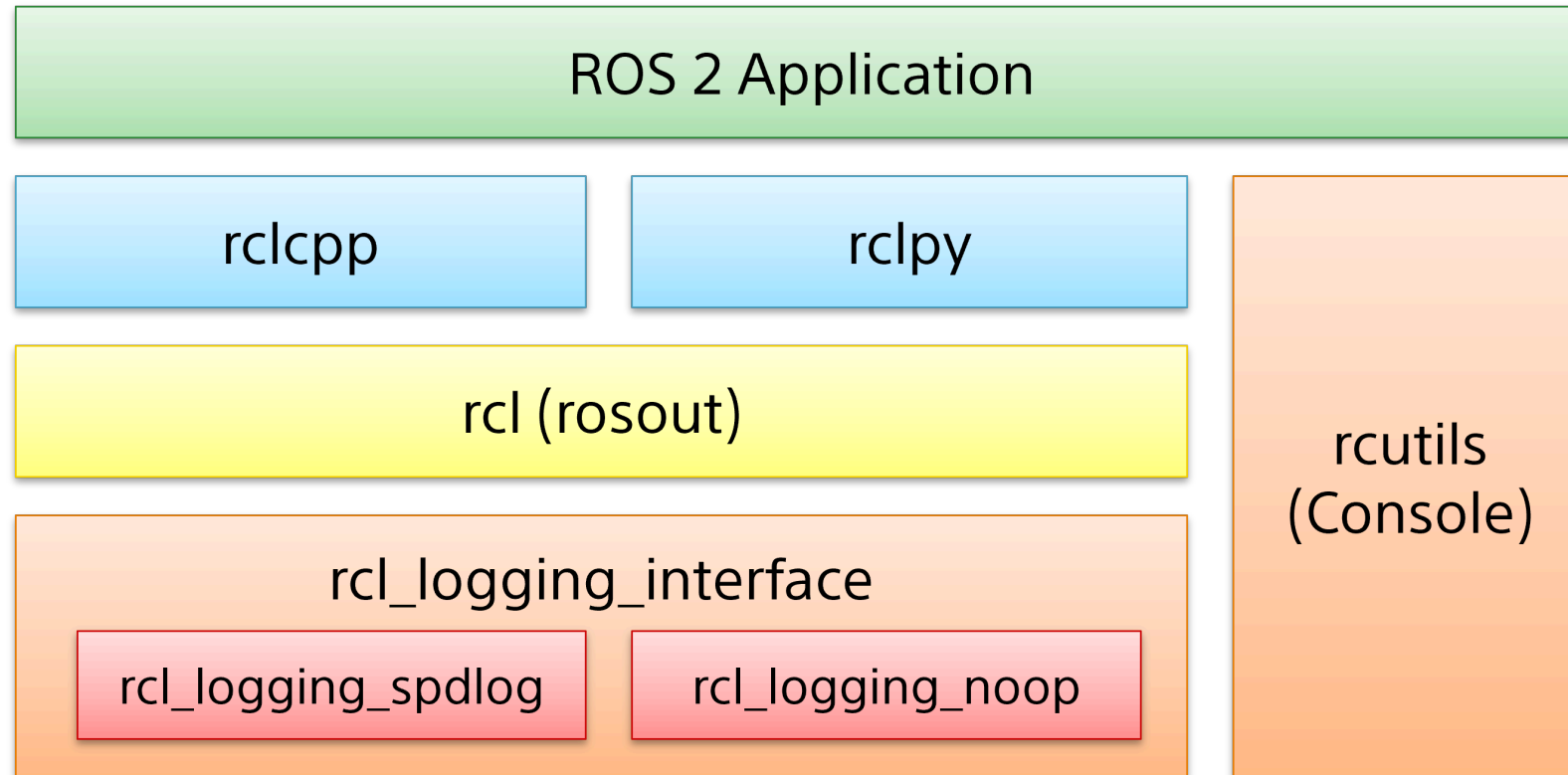


# Actions

 ros2 action



# Logging



- logging subsystem can output the log to console, files and `/rosout` topic.
- `/rosout` topic is to publish and subscribe all logging data in ROS 2 system.
- Multiple severity levels are supported.
- Environmental variables to support log configuration and format.



# More Features

- Quality of Service
- Physical Domain / Discovery Range Control
- ROS 2 Security
- True Zero Copy / Loaned Message
- Wait for Acknowledgement
- Content Filtered Topics
- Micro-Controller Support (Micro-ROS)

# Quality of Service

- QoS policies such as history, depth, reliability, durability, deadline so on.
- QoS preset profiles, e.g) system\_default, sensor\_data and service.
- QoS events, callback to be fired based on the event, i.e) deadline missed, liveliness missed.
- QoS incompatibility can happen between publisher and subscriber.
- MachedEvent can be generated when any publisher and subscription establishes or drops the connection between them.

## **ROS\_DOMAIN\_ID** Partition

- [ROS\\_DOMAIN\\_ID](#), primary mechanism for having different logical networks share a physical network is known as the Domain ID.

## Improved Dynamic Discovery

- Discovery protocol initiated in the same subnet By default.
- User can configure the discovery range with environmental variable, such as only localhost, specific peers.

## ROS 2 Security

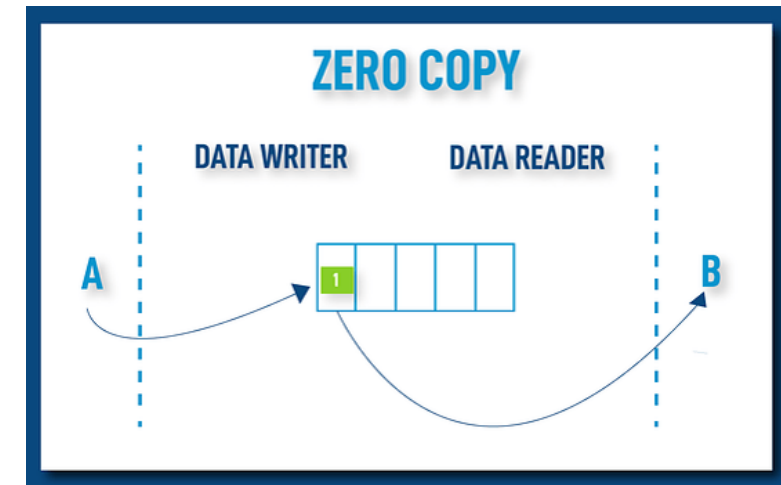
The ability to secure communications among nodes within the ROS 2 computational graph.

Security files enable encryption and authentication, and define policies both for individual nodes and for the overall ROS graph.

see [Setting up security](#) for more details.

# Zero Copy Loaned Message

- True Zero Copy / Write Just Once
- No encapsulation / No Serialization
- Copy-Less communication channel using shared memory.
- Application borrows memory from RMW Implementation.
- Fast-DDS v2.2.0 (ros:galactic or later)



- `LoanedMessage`
  - This class needs to be used by application to borrow memory from middleware during publication.
  - [LoanedMessage Talker Demo](#)
- Constraints
  - bound data type only supported. (plain data type)
- Configuration
  - Suitable for `PREALLOCATED_MEMORY_MODE` and `PREALLOCATED_WITH_REALLOC_MEMORY_MODE` memory configurations only
  - see [MemoryManagementPolicy](#)

| DDS              | Tier | Description                                                                                                                            |
|------------------|------|----------------------------------------------------------------------------------------------------------------------------------------|
| Fast-DDS         | 1    | Zero Copy supported. Even with shared memory transport, DDS feature full supported.                                                    |
| Cyclonedds       | 1    | DDS with shared memory bypass by iceoryx which requires daemon. When using shared memory, DDS feature cannot be supported such as QoS. |
| RTI Connnext DDS | 1    | Commercial only, but most featured DDS implementation                                                                                  |
| Zenoh            | 1    | Not supported                                                                                                                          |

## Wait for Acknowledgements

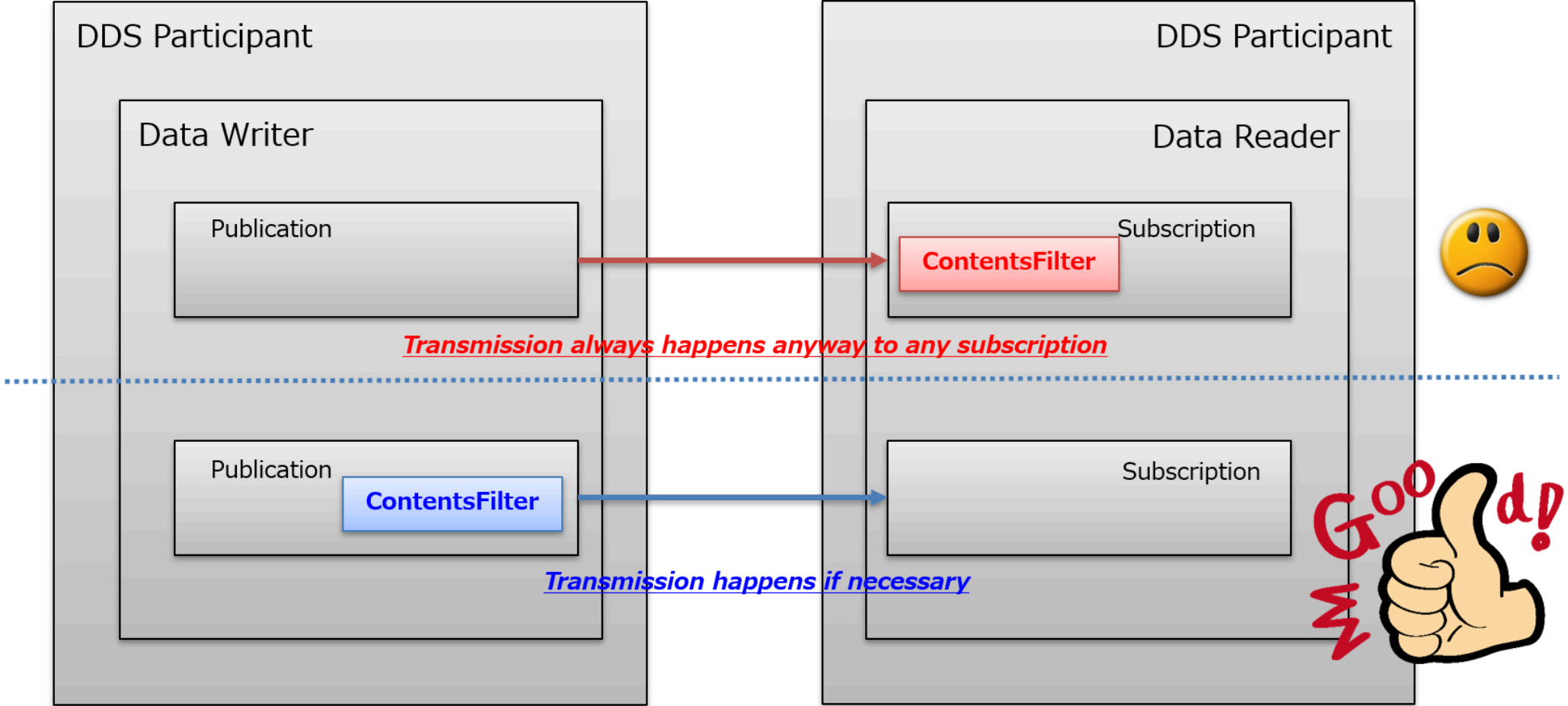
This operation blocks the calling thread until either all data written by the reliable DataWriter entities is acknowledged by all matched reliable DataReader entities, or else the duration specified by the `max_wait` parameter elapses, whichever happens first. A return value of `OK` indicates that all the samples written have been acknowledged by all reliable matched data readers; a return value of `TIMEOUT` indicates that `max_wait` elapsed before all the data was acknowledged.



# Content Filtered Topics

- RMW Content Filtering completed. (Fast-DDS / RTI supported)
- Optimization to conserve network resource.
- [ROS2 Design Overview](#)

| DDS            | Content Filter Supported?      |
|----------------|--------------------------------|
| Fast-DDS       | Fully(Reader/Writer) Supported |
| Cyclonedds     | Not Supported                  |
| RTI Connex DDS | Fully(Reader/Writer) Supported |
| Zenoh          | Not Supported                  |



ContentFilteredTopic provides the improvement for CPU consumption and network traffic.

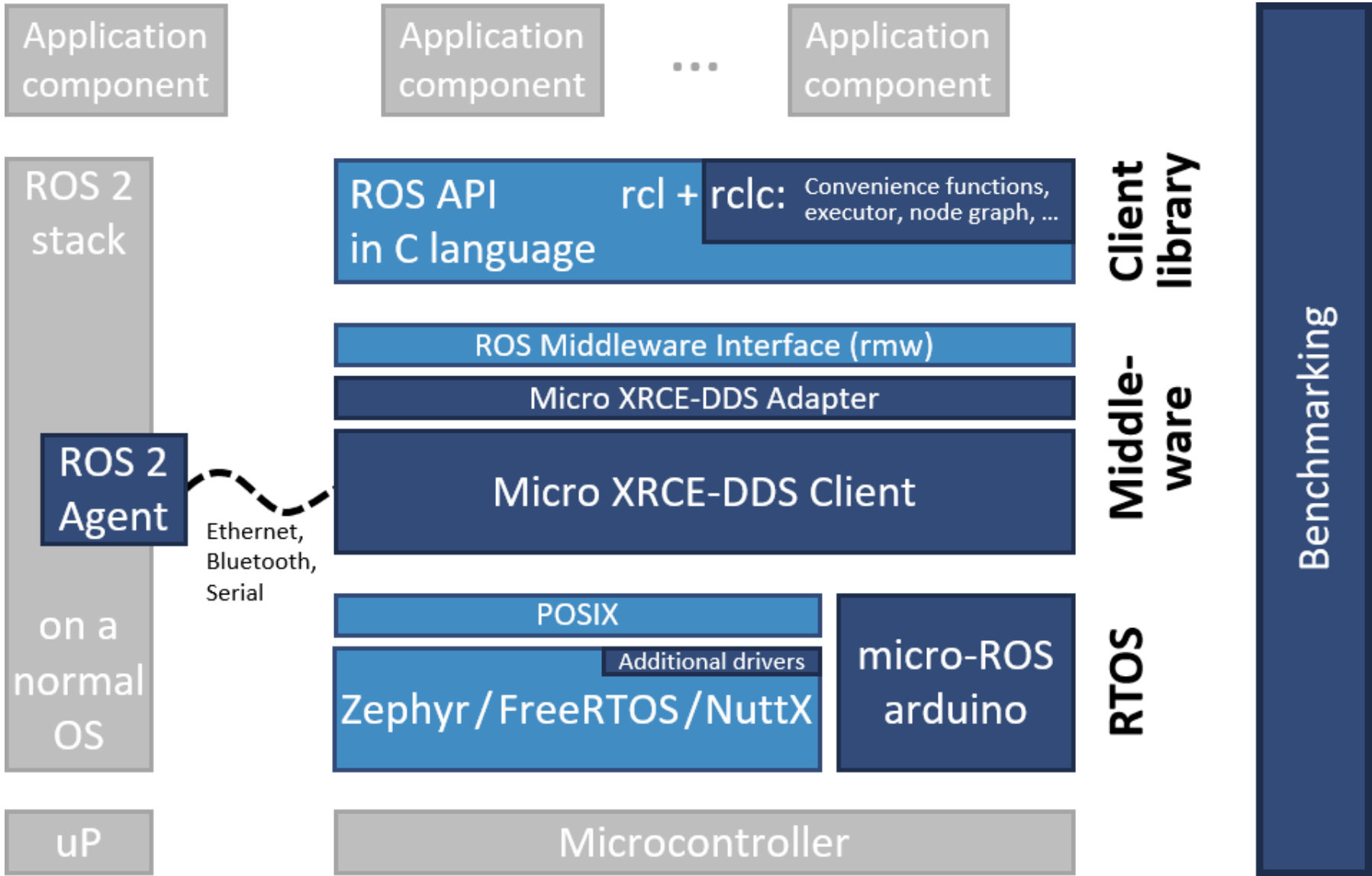
- Reader can get what they need to receive.
- Much less network consumption and user callbacks.
- Application can be agnostic from filtering.
- Filtering expression and parameter is really flexible.
- Filtering can be reconfigured at runtime.

# Micro-Controller Support

*puts ROS 2 onto microcontrollers!*

- Extend ROS 2 for Micro-Controllers / Real-Time OS
- ROS 2 agent proxy required
- eXtremely Resource Constrained Environment DDS
- SPRESENSE w/ NuttX Supported





# Reference

- [ROS 2 Documentation](#)
- [Governance](#)
- [Why ROS 2?](#)
- [ROS 2 Design](#)
- [Micro-ROS](#)